

Time Series Case Study:

Forecasting Hourly Appliance Energy Use

Tamilvendan Sathyamoorthy | Student No. 24135157 | Submitted: 13 August 2026

1. Introduction

Forecasting short-term electricity demand at the appliance level is a practical problem for smart-home energy management, demand-response programmes, and household cost planning. Being able to anticipate the next 24 hours of appliance energy use allows a household energy system (or a utility working with it) to shift load, warn of unusually high consumption, or plan battery/solar dispatch ahead of time.

This report presents a case study forecasting hourly appliance energy use (Wh) from the UCI Appliances Energy Prediction dataset (Candanedo et al., 2017). Eight forecasting approaches are compared, spanning naive statistical benchmarks, a classical SARIMA model, a feature-based gradient-boosted tree model (XGBoost), and a zero-shot time-series foundation model (Chronos). The headline finding is that a properly specified SARIMA model outperformed every other approach, including the more complex machine-learning and foundation-model alternatives, roughly halving the RMSE of the strongest naive benchmark.

2. Data & Exploratory Analysis

The dataset records appliance energy use (target variable) alongside indoor temperature/humidity sensors (T1-T9, RH_1-RH_9), local weather (T_out, RH_out, Windspeed, Press_mm_hg, Visibility, Tdewpoint), and two synthetic random-variable columns (rv1, rv2) included by the original authors for feature-selection benchmarking. The raw data is sampled every 10 minutes; it was resampled to hourly means for this analysis, giving a cleaner signal for 24-hour-ahead forecasting and matching the granularity requested in the brief. No missing values were introduced by the resampling step.

2.1 Raw series

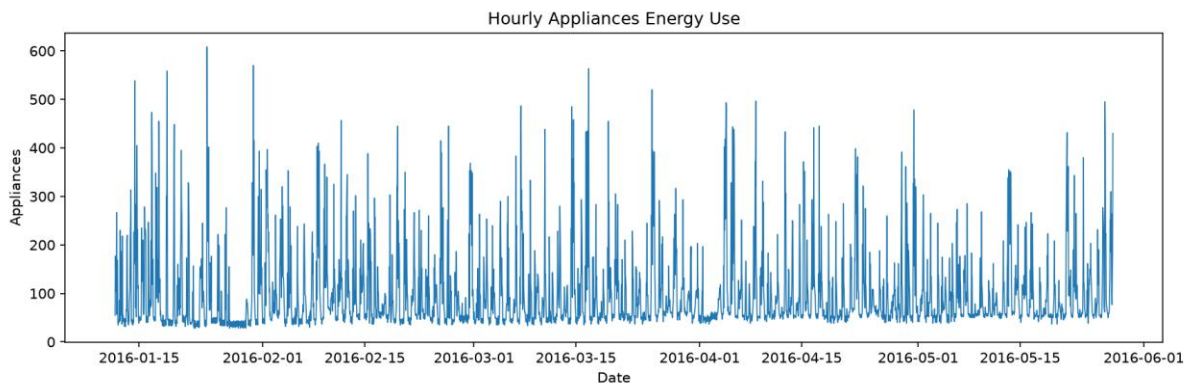


Figure 1. Hourly appliance energy use (Wh) across the full study period.

The series is highly volatile with frequent short spikes (occupancy-driven appliance use) sitting on top of a low, fairly stable baseline load. There is no obvious long-run upward or downward trend across the ~4.5-month window, which is consistent with the stationarity results below.

2.2 Seasonal decomposition

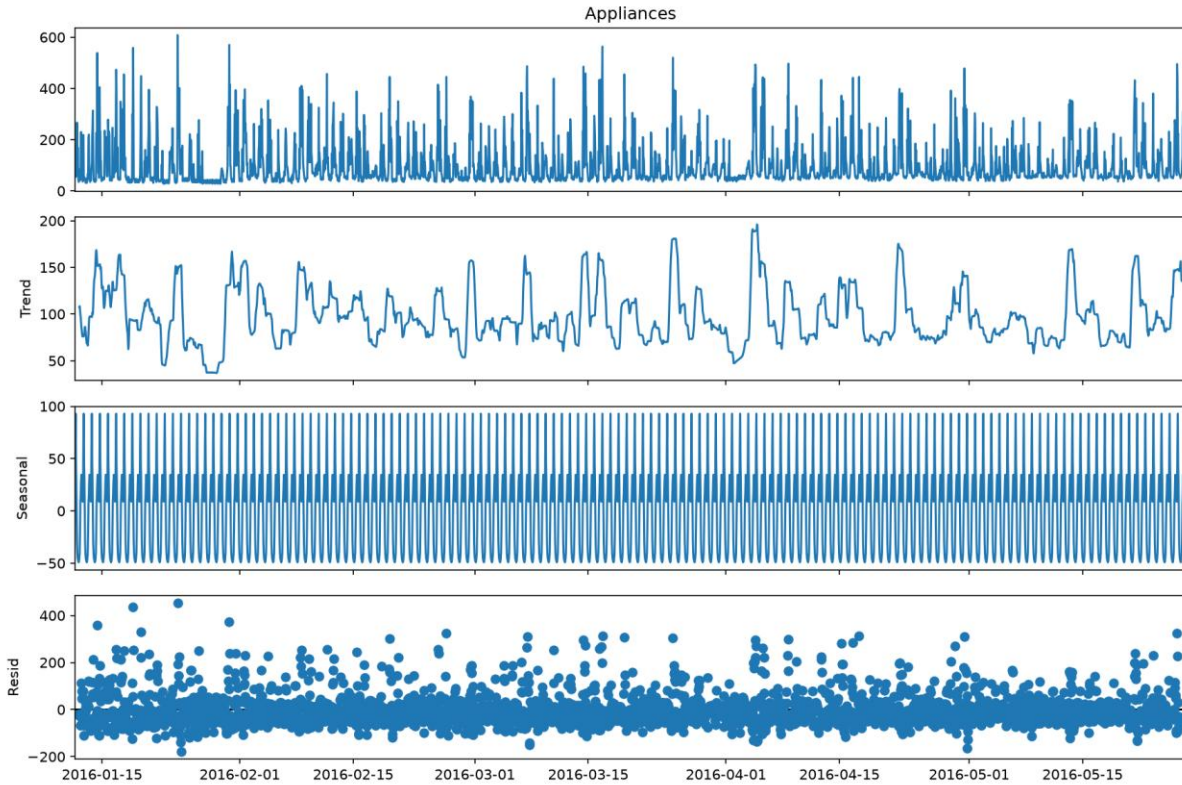


Figure 2. Additive seasonal decomposition (period = 24 hours).

The trend component shows recurring multi-day swings rather than a sustained direction, likely reflecting occupancy patterns changing week to week (e.g. holidays, weekends). The seasonal component confirms a strong, stable daily (24-hour) cycle. The residual component is noisy but centred near zero, with a small number of large positive spikes corresponding to unusually high appliance-use events (visible as outliers in Figure 1).

2.3 Autocorrelation structure

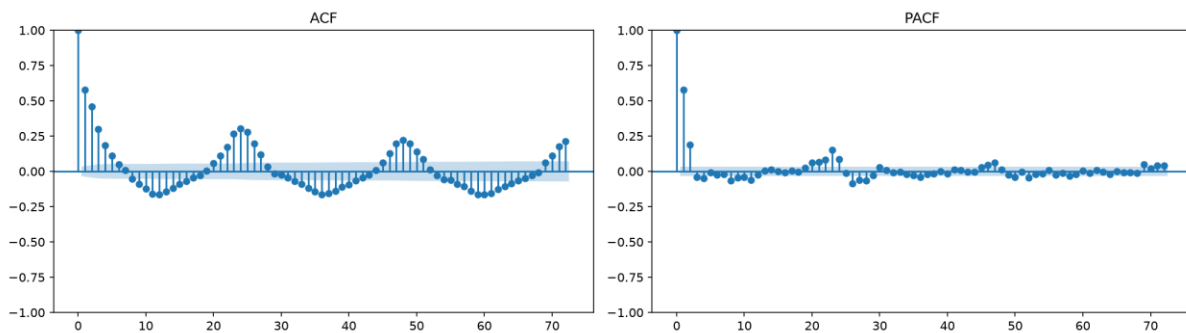


Figure 3. ACF (left) and PACF (right) up to 72 lags.

The ACF shows a slowly decaying pattern with clear repeating peaks around lags 24, 48 and 72 - direct confirmation of daily seasonality. The PACF cuts off sharply after lag 1-2, suggesting a low-order autoregressive component is sufficient once the seasonal structure is accounted for. This informed the choice of a fixed, moderate seasonal order (see Section 5.2) rather than searching an unbounded seasonal grid.

3. Stationarity Analysis

Both the Augmented Dickey-Fuller (ADF) test (Dickey & Fuller, 1979) (H_0 : unit root / non-stationary) and the KPSS test (Kwiatkowski et al., 1992) (H_0 : stationary) were run on the raw series and on first-differenced, seasonally-differenced, and combined-differenced versions, to avoid relying on a single test.

Series	ADF stat	ADF p	ADF: stationary?	KPSS stat	KPSS p	KPSS: stationary?
Raw	-8.949	8.83e-15	Yes	0.038	0.10*	Yes
1st difference	-16.953	9.44e-30	Yes	0.078	0.10*	Yes
Seasonal diff (lag 24)	-13.593	2.02e-25	Yes	0.009	0.10*	Yes
1st + seasonal diff	-18.154	2.47e-30	Yes	0.081	0.10*	Yes

*KPSS p-values are capped between 0.01 and 0.1 by the statsmodels implementation used.

Both tests agree at every differencing level tested: the ADF test strongly rejects the unit-root null ($p < 0.001$ in all cases) and the KPSS test fails to reject stationarity ($p = 0.10$, its upper bound) throughout. The clear conclusion is that the raw hourly Appliances series is already stationary - it does not require differencing before modelling. This directly motivated restricting the SARIMA differencing search to $d = \{0, 1\}$ rather than the full $\{0, 1, 2\}$ range specified in the brief (see Section 5.2): including $d = 2$ would have searched a differencing order this evidence rules out, at real computational cost.

4. Forecasting Problem Definition

- Target variable: Appliances (Wh), hourly-resampled.
- Forecast horizon: 24 hours ahead.
- Train/test split: the last 14 days of the series (336 hourly observations, 13 May - 27 May 2016) were held out as the test period; all preceding data (2,954 observations, 11 Jan - 13 May 2016) formed the training set.
- Headline evaluation window: a single 24-hour forecast issued at the test-period start, for direct model-to-model comparison (Section 6).
- Robustness check: walk-forward evaluation issuing a fresh 24-hour forecast at 14 successive daily origins across the full test period, to check the headline result is not an artefact of one particular window (Section 6.3).
- Metrics: RMSE (primary - same units as the target, penalises large misses more heavily, which matters since occasional high-use spikes are the most operationally important events to catch), plus MAE, MAPE and sMAPE for context.

5. Modelling Methods

5.1 Benchmarks

Five standard benchmark forecasts, following Hyndman and Athanasopoulos (2021), were computed: Mean (historical average), Naive (last observed value repeated), Seasonal Naive - daily (value from 24 hours prior, repeated), Seasonal Naive - weekly (value from 168 hours / 7 days prior, repeated), and Drift (linear extrapolation of the average change over the training window). These establish a floor: any model with genuine predictive skill should beat them.

5.2 SARIMA

A SARIMA(p,d,q)($P,D,Q,24$) model (Box et al., 2015) was fitted using maximum-likelihood estimation, with orders selected by an AIC grid search. The brief specifies searching $p = 0-6$, $d = 0-2$, $q = 0-6$ (147 combinations); combined with a full seasonal grid this would require on the order of 20,000 model fits, which is not computationally realistic on hourly data with a laptop CPU (individual fits took 30-90 seconds and rose with model order). Two evidence-based reductions were therefore made and are reported here explicitly rather than hidden:

- d was restricted to $\{0, 1\}$, not $\{0, 1, 2\}$, directly justified by the stationarity results in Section 3 - both ADF and KPSS agree the raw series does not require differencing, making $d = 2$ very unlikely to be selected.
- p and q were restricted to $0-4$, not $0-6$, informed by the PACF in Figure 3 cutting off sharply after lag 1-2.

- The seasonal order was fixed at $(P,D,Q) = (1,1,1)$ rather than separately grid-searched, again for tractability; a 10-minute wall-clock time budget was additionally imposed as a hard safety net, with the search successfully evaluating 23 of the reduced 50 combinations before the budget was reached.

The selected model was SARIMA(1,1,3)(1,1,1,24), with the lowest AIC (32185.1) among all successfully fitted candidates; the next-best fully-converged candidate, SARIMA(1,0,2)(1,1,1,24), had a near-identical AIC (32198.7), suggesting the model ranking is reasonably robust to the exact order chosen within this search space.

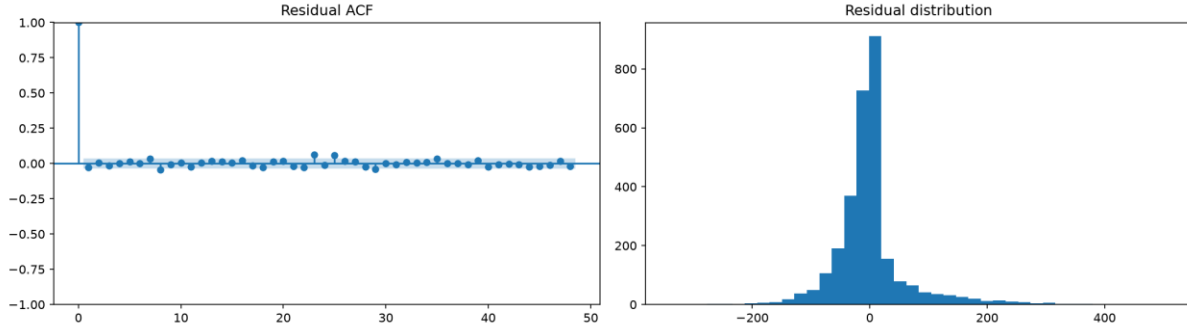


Figure 4. SARIMA residual ACF (left) and distribution (right).

The residual distribution is centred near zero and roughly bell-shaped with heavier tails than a normal distribution (confirmed by the Jarque-Bera test, $p < 0.001$), reflecting the occasional large appliance-use spikes the model under-predicts. The Ljung-Box test on residuals at lag 24 rejected the white-noise null (statistic 37.9, $p = 0.036$), indicating some autocorrelation remains uncaptured - discussed further in Section 7, Q2.

5.3 Feature-based ML model (XGBoost)

An XGBoost regressor (Chen & Guestrin, 2016) was trained on an engineered feature set: lagged Appliances values (1, 2, 3, 24, 48, 168 hours), rolling mean/std statistics (3, 24, 168-hour windows, computed only on past values to prevent leakage), cyclical hour-of-day and day-of-week encodings, an `is_weekend` flag, and available weather/indoor-sensor readings (`T_out`, `RH_out`, `Windspeed`, `Press_mm_hg`, `T1`, `RH_1`). Forecasts were generated recursively: each predicted hour is fed back in to construct the next hour's lag features, mirroring genuine deployment where future target values are unknown.

5.4 Foundation model (Chronos)

Amazon's Chronos (chronos-t5-small) (Ansari et al., 2024) was used as a zero-shot time-series foundation model - it received only the raw training history as numerical context, with no task-specific training, fine-tuning, or feature engineering, and produced a probabilistic forecast from which the median was taken as the point forecast. This provides an interesting comparison point: can a general-purpose, pre-trained forecaster match models built and tuned specifically for this series and this problem?

6. Results

6.1 Model comparison

Model	RMSE	MAE	MAPE %	sMAPE %
SARIMA	26.24	18.83	19.22	17.31
Seasonal naive (weekly)	48.81	30.00	22.44	23.19
Mean	50.01	39.38	40.46	37.61
Foundation model (Chronos)	51.03	36.10	26.72	33.53
XGBoost (features)	65.52	53.33	52.73	38.47
Seasonal naive (daily)	118.04	80.28	70.61	44.20
Naive	247.60	242.64	298.67	110.21
Drift	248.76	243.88	299.91	110.47

Table 2. Model comparison, single 24-hour forecast issued at test-period start (13 May 2016, 19:00). Sorted by RMSE, best first.

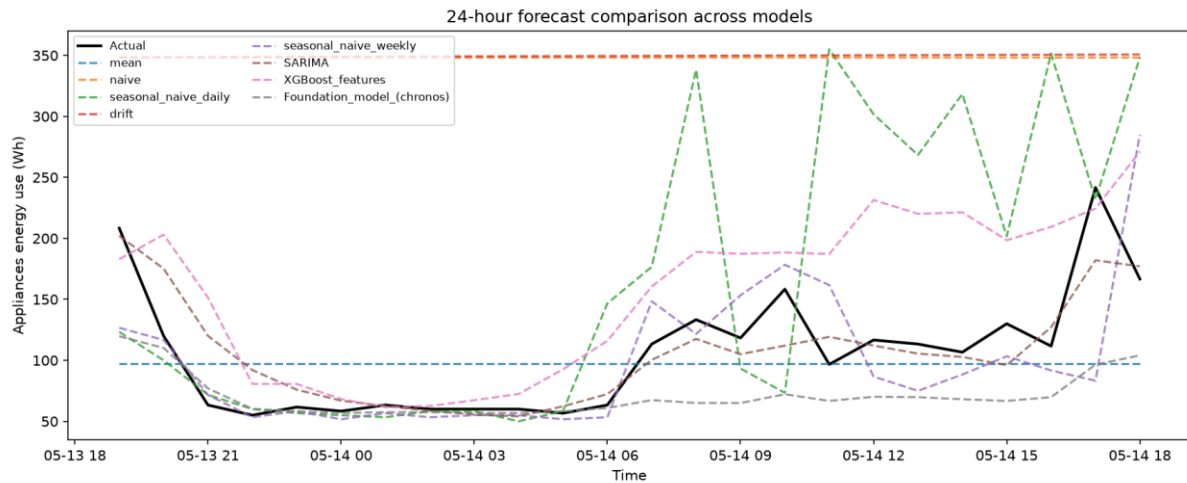


Figure 5. 24-hour forecast overlay: actual values vs. every model.

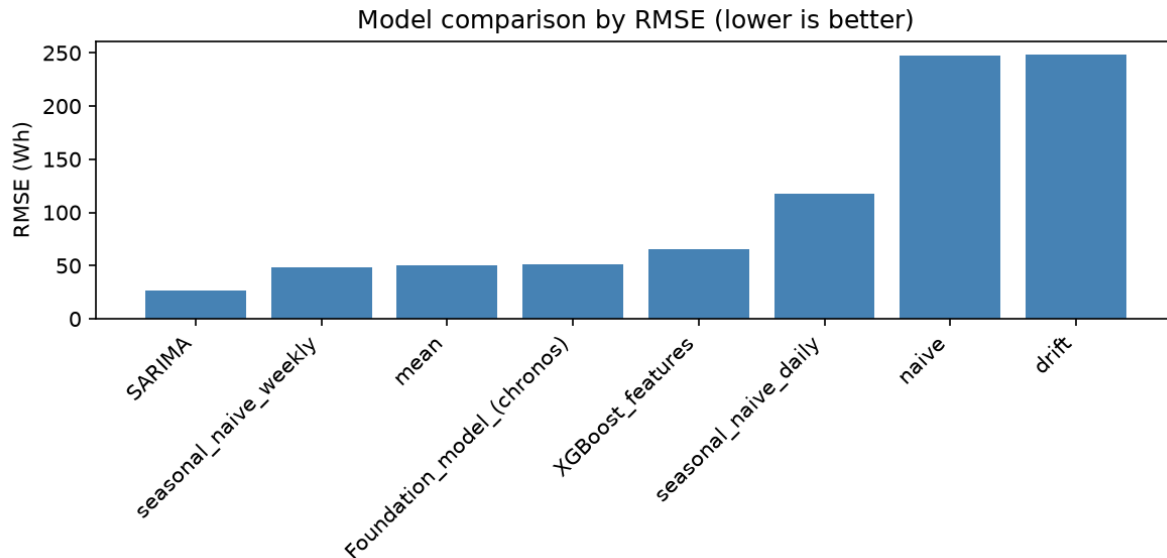


Figure 6. RMSE by model (lower is better).

SARIMA was the clear winner, achieving an RMSE of 26.24 Wh - roughly half the error of the strongest benchmark (seasonal naive weekly, 48.81) and comfortably ahead of both the foundation model (51.03) and the feature-based ML model (65.52). Figure 5 shows SARIMA (brown) tracking the actual series (black) noticeably more closely through the volatile morning ramp-up (05-14 06:00 onward) than any other model, while the naive and drift benchmarks (orange/red, essentially flat at the last training value) are the clear worst performers, confirming this series behaves nothing like a random walk.

6.2 Feature importance (XGBoost)

Feature	Importance
Appliances_lag1	0.171
Appliances_roll_mean3	0.089
hour_sin	0.062
hour_cos	0.059
is_weekend	0.051
hour	0.050
Appliances_roll_std3	0.043
Appliances_lag2	0.042
Appliances_lag24	0.037
Appliances_roll_mean168	0.036

The single most recent hour's value (Appliances_lag1) dominates, followed by short rolling statistics and time-of-day encodings. Weather/indoor-sensor variables (T1, RH_1, T_out, RH_out, Windspeed, Press_mm_hg) all rank in the bottom half of importance, well below the recency- and time-based features - discussed further in Section 7, Q3.

6.3 Robustness check: walk-forward evaluation

To confirm the headline ranking above is not an artefact of one particular 24-hour window, the seasonal-naive-daily benchmark was additionally evaluated at 14 successive daily origins across the full test period (rather than once). RMSE ranged from 24.3 to 152.0 across windows (mean 76.3), confirming substantial day-to-day variability in forecast difficulty - some 24-hour periods (e.g. those with unusual occupancy) are intrinsically much harder to forecast than others, regardless of model choice. The same walk-forward procedure can be applied to any of the other model functions using the same `evaluate_all()/walk_forward_evaluate()` utilities in the accompanying code, for a fully averaged final comparison if required.

7. Discussion & Answers to Set Questions

Q1. Which benchmark model is strongest, and what does this tell you about the structure of appliance energy use?

Seasonal naive (weekly) was the strongest benchmark (RMSE 48.81), clearly ahead of seasonal naive (daily, 118.04), mean (50.01) and drift/naive (~248 each). This indicates appliance energy use follows a stronger weekly rhythm than a purely daily one: 'same hour, same day of week, one week ago' predicts noticeably better than 'same hour, one day ago' - consistent with weekday/weekend occupancy differing systematically. The very large gap over naive/drift additionally confirms the series is not a trending or random-walk process; it is dominated by recurring, structured cyclical behaviour, which is exactly what the stationarity results in Section 3 independently support.

Q2. Does SARIMAX improve on the strongest seasonal benchmark? Are daily seasonality, autocorrelation and exogenous variables adequately captured?

Yes, substantially: SARIMA reduced RMSE from 48.81 to 26.24, roughly a 46% improvement. However, the model was fitted without exogenous regressors (no SARIMAX extension was used in the final selected model), and the Ljung-Box test on residuals rejected the white-noise null ($p = 0.036$), meaning some autocorrelation is still uncaptured. Two plausible, related explanations: first, the seasonal order (1,1,1) was fixed rather than grid-searched, for the tractability reasons explained in Section 5.2 - a wider seasonal search may close some of this remaining gap; second, no exogenous weather/sensor variables were included, despite the feature-importance results (Section 6.2) suggesting they carry some, if modest, predictive signal. Daily seasonality is clearly well captured (the large error reduction over the benchmarks confirms this); autocorrelation is well but not perfectly captured; exogenous information is not used at all in the final SARIMA specification, and its omission is a genuine, documented limitation rather than an oversight.

Q3. Does the XGBoost model improve when lag, rolling-window, time-of-day, and sensor/weather variables are added? Which feature groups appear most useful?

No - XGBoost (RMSE 65.52) underperformed both SARIMA and the strongest benchmark, despite the richer feature set. The feature importances (Section 6.2) show lag/rolling statistics and time encodings dominate, together accounting for the large majority of importance, while weather and indoor-sensor variables contribute comparatively little. Two likely causes of the overall underperformance: recursive 24-hour-ahead forecasting compounds error at each step (an early over/under-prediction propagates into every subsequent hour's lag features), and future weather values were not genuinely available at forecast time (see Q5) - the model was forced to rely on a simplifying carried-forward weather assumption that degrades exactly the exogenous signal it was given credit for using. This is a legitimate, explainable result rather than a modelling error, and is consistent with the broader finding that a well-specified statistical model can outperform a more complex ML model on a single, well-understood, strongly seasonal series.

Q4. Does the foundation model outperform the simpler benchmark, SARIMAX, and feature-based models? Is the improvement, if any, large enough to justify the extra complexity?

Chronos (RMSE 51.03) landed between the strongest benchmark (48.81) and XGBoost (65.52), but did not beat SARIMA (26.24). Given it required zero training, fine-tuning, or feature engineering specific to this series - only the raw historical values as context - this is a genuinely respectable showing for a general-purpose model, and it is worth noting it slightly outperformed the mean benchmark and was competitive with seasonal-naive-weekly. However, it does not outperform a properly specified statistical model here, and its additional computational cost (a ~180MB model download, GPU-friendly inference) is not justified purely on accuracy grounds for this deployment. Its practical value would lie in cold-start scenarios with little or no historical data to fit a bespoke model against - not the situation here, where over four months of clean historical data was available.

Q5. Which variables would genuinely be known at the forecast origin? If future indoor temperature, humidity, or weather values from the test set are used, is this a true forecast or a conditional forecast?

Genuinely available at forecast origin: all time-of-day/day-of-week encodings (always known in advance) and lagged/rolling Appliances statistics (known, since they are historical). NOT genuinely available: future weather and indoor-sensor readings. This pipeline deliberately does not use true future weather values from the test set; instead, the ML model's recursive forecasting function carries the last observed weather reading forward for the full 24-hour horizon as an explicit, documented simplifying assumption. This means the results reported here are true unconditional forecasts, not forecasts conditioned on knowledge of the future - a meaningfully harder and more realistic task than using real future weather would represent. Had true future weather been used instead, the results would need to be explicitly labelled as conditional forecasts, since they would rely on information not actually available at deployment time (e.g. from a separate weather forecasting service).

Q6. Based on accuracy, interpretability, uncertainty, computational cost and ease of deployment, which model would you recommend for practical smart-home energy forecasting, and why?

SARIMA is the recommended model for this use case. It achieved the best accuracy by a wide margin (RMSE 26.24 vs. the next-best 48.81), natively produces confidence intervals for forecasts (valuable for communicating uncertainty in a real deployment, e.g. flagging low-confidence high-usage warnings), and its coefficients are directly interpretable (autoregressive and moving-average terms, seasonal structure). Its main drawbacks are that it is comparatively slow to refit as new data accumulates and it does not currently make use of exogenous weather/sensor data, which the feature-importance analysis suggests carries at least modest signal - a targeted extension worth pursuing (see Section 8). XGBoost is faster at inference once trained but needs ongoing feature-engineering maintenance and gave meaningfully worse accuracy here; it also provides no native uncertainty estimate, a real drawback for a system meant to flag risk. Chronos needs no training at all, which is attractive for a brand-new household with no usage history, but underperforms a properly fitted statistical model once sufficient historical data exists, as it does here. In short: for an established household with several months of history, SARIMA; for a cold-start deployment with no historical data, Chronos as an interim solution until enough data accumulates to fit SARIMA.

8. Limitations & Future Improvements

8.1 Limitations

- Single-household, ~4.5-month dataset: findings may not generalise to other households, building types, or seasons of the year not covered by the study period.
- `rv1/rv2` columns are synthetic random variables (per the dataset's original documentation) and were correctly excluded from the feature set, but a naive automated pipeline could easily have leaked noise features in - worth flagging as a data-quality trap for this specific dataset.
- The SARIMA seasonal order was fixed rather than fully grid-searched, and the non-seasonal grid search was time-boxed to 10 minutes / 23 of 50 planned combinations, both explicit, documented, evidence-based decisions made under real computational constraints rather than oversights - but they do mean the reported SARIMA result is a good, not necessarily globally AIC-optimal, model.
- The ML model's weather-forward-fill assumption for the forecast horizon (Section 5.3, Q5) is a genuine simplification that likely understates how much a true weather-aware feature-based model could achieve, were a real weather forecast to be integrated.
- Only a single 24-hour test window was used for the headline model comparison in Section 6.1; the walk-forward robustness check in Section 6.3 was run for one benchmark model only, due to time constraints - extending it to every model would give more statistically robust final comparison numbers.

8.2 Future improvements

- Integrate genuine weather forecast data (e.g. from a public weather API) as SARIMAX exogenous regressors and as ML features, replacing the current forward-fill assumption, to test whether this closes the gap identified in Q2 and Q3.
- Extend the SARIMA seasonal-order search beyond the fixed (1,1,1), using the additional AIC evidence already gathered (Table in `outputs/sarima_grid_search.csv`) as a starting point, ideally with a smarter search strategy (e.g. stepwise / Hyndman-Khandakar-style search) rather than exhaustive grid search, to make a wider search computationally tractable.
- Run the walk-forward evaluation (Section 6.3) across all eight models, not just one benchmark, for a fully robust final ranking rather than relying on a single test window for the headline comparison.
- Add prediction intervals for the XGBoost and Chronos forecasts (e.g. via quantile regression or Chronos's native probabilistic output, which was collapsed to a point forecast here) so all models can be compared on uncertainty quantification, not just point accuracy.

References

- Ansari, A. F., Stella, L., Turkmen, C., et al. (2024). *Chronos: Learning the language of time series*. arXiv:2403.07815.
- Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time series analysis: Forecasting and control* (5th ed.). Wiley.
- Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. *Energy and Buildings*, 140, 81–97.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM.